

FIXPOINT SWARM-R

Phase 5: Intelligence Substrate MCCE · ECLS · ISLS

From Trading Instrument to
Autonomous DeFi Intelligence Platform

Input: Phase 4 repository (18 crates, DSHAE, GUI, sandbox)

Additions: Mycelial Crypto-Cartography Engine (MCCE):
persistent topological learning graph
Emergent Constraint Lattice Spectroscopy (ECLS):
passive market constraint discovery
Intelligent Semantic Ledger Substrate (ISLS):
unifying persistence and consensus layer

New crates: `fsr-mcce`, `fsr-ecls`, `fsr-isls`

Total crates: 21 (18 existing + 3 new)

Date: 12 March 2026

Prime Directive

Phase 5 transforms FIXPOINT SWARM-R from a trading instrument that reacts to markets into an **intelligence platform that understands markets**. MCCE builds a permanent, self-expanding map of the crypto ecosystem. ECLS reads the constraint programs that markets enforce. ISLS provides the append-only semantic ledger that binds everything into a replay-verifiable, structurally grounded knowledge substrate. After Phase 5, the system accumulates intelligence over months and years — the longer it runs, the more it sees, the better it trades.

Contents

1	Phase 5 Scope	2
1.1	The Three Systems	2
1.2	What Phase 5 Does NOT Add	2
2	ISLS: Intelligent Semantic Ledger Substrate	2
2.1	New Crate: <code>fsr-isls</code>	2
2.2	Core Types	3
2.3	Storage Tiers	3
2.4	Consensus Functional	4
3	MCCE: Mycelial Crypto-Cartography Engine	4
3.1	New Crate: <code>fsr-mcce</code>	4
3.2	Mycelial HDAG	4
3.3	MCCE Layers (Simplified for Phase 5)	5
3.4	MCCE Learning Dynamics	6
4	ECLS: Emergent Constraint Lattice Spectroscopy	6
4.1	New Crate: <code>fsr-ecls</code>	6
4.2	Constraint Templates	7
4.3	Constraint Scanner	7
4.4	Lattice Crystal	8
4.5	DSHAE Integration	8
5	GUI Additions	9
6	Sandbox Extensions	9
7	Integration Architecture	10
8	Configuration	10
9	Implementation Order	11
10	New Dependencies	12
11	Definition of Done (Phase 5)	12
12	Phase 6 Preview (Non-Normative)	12

1 Phase 5 Scope

1.1 The Three Systems

Layer	System	Role	Metaphor
L5	ECLS	Constraint discovery dome	Spectroscopist reading molecular bonds
L4	MCCE	Persistent topological map	Mycelial network spreading through soil
L3	ISLS	Semantic ledger substrate	Bedrock that remembers everything
L2	DSHAE	Holographic arbitrage signals	Navigator reading the map
L1	FSR Engine	Execution infrastructure	Ship's engine room
L0	Market	Reality	The ocean

Information flow: Market → FSR observation → ISLS persistence → MCCE graph building → ECLS constraint reading → DSHAE signal generation → FSR execution. Each layer reads from below and writes semantic crystals upward.

1.2 What Phase 5 Does NOT Add

- No on-chain interaction (DeFi protocol interaction is Phase 6+).
- No additional exchanges beyond Binance + Kraken.
- No web GUI (desktop egui only).
- No ML/AI models (topology is the model — MCCE Principle P2).

2 ISLS: Intelligent Semantic Ledger Substrate

ISLS is the unifying persistence and consensus layer. It replaces the ad-hoc chain/snapshot persistence from Phase 2 with a formal, layered substrate.

2.1 New Crate: fsr-isls

Listing 1: fsr-isls module structure

```
fsr-isls/
src/
  lib.rs           // public API
  observation.rs   // L0: observation canonicalization
  persistence.rs   // L1: append-only graph storage
  constraint.rs     // L2: constraint extraction interface
  consensus.rs     // L3: resonant consensus + commit
  adaptation.rs    // L4: morphogenic adaptation hooks
  crystal.rs       // Semantic Crystal artifact
  evidence.rs      // Evidence chain (replaces fsr-chain internals)
  storage.rs       // Tiered storage (hot/warm/cold)
  config.rs
```

2.2 Core Types

Listing 2: ISLS core types

```

/// Canonical observation (ISLS Definition 4.1)
pub struct Observation {
  pub time: u64,           // microseconds
  pub source: SourceId,    // which venue/chain/feed
  pub provenance: Provenance, // how we got this data
  pub payload: ObsPayload, // the actual data
  pub context: MeasureContext, // measurement conditions
  pub digest: Hash256,
}

/// Persistent structural graph (ISLS Definition 4.4)
pub struct PersistentGraph {
  pub vertices: BTreeMap<EntityId, VertexRecord>,
  pub edges: Vec<TypedEdge>,
  pub embeddings: BTreeMap<EntityId, [Q32; 5]>, // Phi: V -> H5
  pub tensor_archive: TensorArchive, // Lambda: historical embeddings
  pub edge_annotations: BTreeMap<EdgeId, FeatureVector>, // Sigma
}

/// Semantic Crystal (ISLS Definition 4.6)
pub struct SemanticCrystal {
  pub id: Uuid,
  pub condensed_region: SubGraph, // G*
  pub constraint_program: Vec<ConstraintId>, // K*
  pub stability: Q32, // F(C)
  pub topo_signature: TopoSignature, // tau(C)
  pub betti: Vec<u32>, // beta(C)
  pub digest: Hash256, // h(C)
  pub evidence: EvidenceChain, // E(C)
  pub commit_proof: CommitProof, // pi(C)
  pub created_at: u64,
}

/// Tripolar state (ISLS Definition 4.3)
pub struct TripolarState {
  pub psi: Q32, // coherence amplitude [0,1]
  pub rho: Q32, // density [0,1]
  pub omega: Q32, // phase frequency
}

```

2.3 Storage Tiers

Tier	Data Age	Format	Location
Hot	< 1 hour	In-memory + bincode	RAM + data/isls/hot/
Warm	1h – 7 days	Bincode on disk	data/isls/warm/
Cold	> 7 days	Zstd-compressed	data/isls/cold/

All tiers are append-only and queryable. Compaction moves data between tiers without logical deletion (ISLS Axiom 3.3). The existing `fsr-chain` shadow/commit chain is migrated to use ISLS storage as its backend.

2.4 Consensus Functional

Listing 3: Resonant consensus (ISLS Section 9)

```
pub fn resonant_consensus(
    crystal: &SemanticCrystal,
    graph: &PersistentGraph,
    config: &ConsensusConfig,
) -> CommitDecision {
    // Consensus = stability * coherence * evidence_completeness
    let stability = crystal.stability;
    let coherence = compute_local_coherence(crystal, graph);
    let evidence = crystal.evidence.completeness_score();

    let score = stability * coherence * evidence;

    if score >= config.commit_threshold {
        CommitDecision::Commit(CommitProof::new(score, crystal))
    } else {
        CommitDecision::Defer
    }
}
```

ISLS Integration Rule

After Phase 5, all system persistence flows through ISLS. The existing **fsr-chain** crate becomes a thin wrapper delegating to **fsr-isls::evidence**. Existing chain semantics (shadow + commit, hash-linking, deterministic replay) are preserved exactly.

3 MCCE: Mycelial Crypto-Cartography Engine

MCCE maintains a permanent, self-expanding graph of the crypto ecosystem. It is the long-term memory of the platform.

3.1 New Crate: **fsr-mcce**

Listing 4: fsr-mcce module structure

```
fsr-mcce/
src/
  lib.rs           // public API
  hdag.rs          // Mycelial HDAG graph operations
  vertex.rs        // Vertex types, 5D embedding
  edge.rs          // Edge types, annotations, lifecycle
  spore.rs         // L0: data ingestion, auto-discovery
  hypha.rs         // L1: pairwise correlation engine
  mycelium.rs      // L2: HDAG construction, hypercube embedding
  fruiting.rs      // L3: signal emission to DSHAE/ECLS
  substrate.rs     // L4: persistent storage (delegates to ISLS)
  basket_advisor.rs // dynamic basket recommendation
  config.rs
```

3.2 Mycelial HDAG

Listing 5: HDAG core

```

pub struct MycelialHdag {
  pub graph: PersistentGraph, // from ISLS
  pub vertex_types: HashMap<EntityId, VertexType>,
  pub correlation_cache: CorrelationCache,
  pub tick: u64,
}

pub enum VertexType {
  Protocol { chain: String }, // Bitcoin, Ethereum, Solana
  Token { symbol: String, chain: String },
  Exchange { name: String, kind: ExchangeKind },
  DeFiPool { protocol: String, pair: String },
  Aggregate { name: String },
}

impl MycelialHdag {
  /// Embed vertex into H5 (MCCE Definition 2.2)
  pub fn embed_vertex(&self, v: EntityId) -> [Q32; 5] {
    let vr = &self.graph.vertices[&v];
    [
      vr.market_cap_log_norm, // x: structural size
      vr.volume_24h_log_norm, // y: structural activity
      vr.volatility_30d_norm, // z: structural risk
      vr.local_coherence, // psi: neighborhood coherence
      vr.phase_at_last_change, // omega: temporal rhythm
    ]
  }

  /// Add or update a correlation edge (Hypha)
  pub fn update_hypha(
    &mut self, u: EntityId, v: EntityId,
    correlation: Q32, causality: Q32, window: u64,
  ) {
    // Create edge if new, update weight if existing
    // Edge weight decays over time (Appendix A.1)
    // Store in ISLS persistence
  }

  /// Get all triangulations over a vertex neighborhood
  pub fn local_triangulations(&self, v: EntityId, depth: u8)
    -> Vec<Triangle>
  {
    // Extract N-hop neighborhood, triangulate
  }
}

```

3.3 MCCE Layers (Simplified for Phase 5)

L	Name	Phase 5 Implementation
L0	Spore	Reads from existing Binance/Kraken venue adapters. Discovers new trading pairs automatically when venues report them. Creates vertex for each new token/pair.

L1	Hypha	Computes rolling pairwise correlations (Pearson on log-returns, window 100 ticks). Stores as edge annotations. Prunes edges below $\rho_{\min} = 0.3$.
L2	Mycelium	Embeds all active vertices into H_5 . Runs TTCP triangulation on the vertex cloud. Detects persistent topological features (clusters, loops).
L3	Fruiting	Emits signals to DSHAE: “these vertices form a stable cluster” or “this triangle has persistent non-transitivity.” These are MCCE’s gifts to the navigator.
L4	Substrate	Delegates to ISLS. All graph state is in the PersistentGraph.

3.4 MCCE Learning Dynamics

Listing 6: What MCCE learns over time

```
// After 1 day: ~50 vertices, ~200 edges, basic correlations
// After 1 week: ~100 vertices, ~1000 edges, stable clusters visible
// After 1 month: ~200 vertices, ~5000 edges, causality patterns
// After 3 months: ~300 vertices, ~15000 edges, regime signatures
// After 1 year: ~500+ vertices, ~50000+ edges, deep structural memory

// Leverage: N vertices -> O(N^2) pairwise, O(N^3) triangulations
// At N=300: 45,000 pairs, 4.5M triangles, rich cascade fodder
```

MCCE Leverage Principle

MCCE does not trade. It accumulates structural knowledge. Every month of operation makes the DSHAE’s signals more precise, because DSHAE reads from an increasingly rich map. This is the “patience as leverage” principle: the system gets better not by training, but by *existing longer*.

4 ECLS: Emergent Constraint Lattice Spectroscopy

ECLS is the analytical dome that reads constraint programs from the MCCE graph.

4.1 New Crate: fsr-ecls

Listing 7: fsr-ecls module structure

```
fsr-ecls/
src/
  lib.rs           // public API
  constraint.rs    // Constraint candidate types
  templates.rs     // 7 constraint template families
  scanner.rs       // Scan HDAG for constraint instantiations
  lattice.rs       // Constraint lattice construction
  inverse_weave.rs // Inverse weaving algorithm
  thermodynamics.rs // Free energy, reaction pathways
  crystal.rs       // Lattice Crystal artifact
  signal.rs        // Signal emission to DSHAE
  config.rs
```

4.2 Constraint Templates

Listing 8: The 7 constraint template families

```
pub enum ConstraintTemplate {
    ///  $|\sigma_j - \mu_j| \leq k * \text{std}_j$ 
    Band { axis: usize, k_sigma: Q32 },

    ///  $|\sigma_i / \sigma_j - c| \leq \text{epsilon}$ 
    Ratio { axis_i: usize, axis_j: usize, ratio: Q32, tolerance: Q32 },

    ///  $|\rho_{uv} - \rho_0| \leq \text{delta}$ 
    Correlation { rho_target: Q32, delta: Q32 },

    ///  $\tau_{uv} \geq \tau_{\text{min}}$  (Granger-like causality)
    Granger { min_lag_score: Q32 },

    ///  $\kappa_{uv}(f) \geq \kappa_{\text{min}}$  (spectral coherence at frequency  $f$ )
    Spectral { frequency_bin: u16, min_coherence: Q32 },

    ///  $\beta_k(\text{VR\_epsilon}) = b$  (topological invariant)
    Topological { betti_dim: u8, expected_value: u32 },

    ///  $|\omega_u - \omega_v - \text{delta}| \leq \text{delta\_omega}$  (phase lock)
    PhaseLock { phase_offset: Q32, tolerance: Q32 },
}

pub struct ConstraintCandidate {
    pub id: Uuid,
    pub template: ConstraintTemplate,
    pub entities: Vec<EntityId>, // which vertices this binds
    pub satisfaction_rate: Q32, // alpha: fraction of window satisfying
    pub window_size: u64, // how long we've observed it
    pub stability: Q32, // does it persist?
    pub first_seen: u64,
    pub last_confirmed: u64,
}
```

4.3 Constraint Scanner

Listing 9: Scanner logic

```
pub fn scan_for_constraints(
    hdag: &MycelialHdag,
    templates: &[ConstraintTemplate],
    config: &EclsConfig,
) -> Vec<ConstraintCandidate> {
    let mut candidates = Vec::new();

    // For each pair (u, v) in the HDAG:
    for (u, v, edge) in hdag.graph.edges.iter() {
        // For each template:
        for template in templates {
            // Evaluate: does this pair satisfy this template
            // over the rolling window?
            let satisfaction = evaluate_template(
                template, u, v, hdag, config.window_size
            );
        }
    }
}
```

```

        if satisfaction >= config.alpha_min {
            candidates.push(ConstraintCandidate {
                template: template.clone(),
                entities: vec![*u, *v],
                satisfaction_rate: satisfaction,
                // ...
            });
        }
    }
}
candidates
}

```

4.4 Lattice Crystal

When ECLS identifies a stable set of mutually consistent constraints, it emits a Lattice Crystal:

Listing 10: LatticeCrystal

```

pub struct LatticeCrystal {
    pub id: Uuid,
    pub constraints: Vec<ConstraintCandidate>,
    pub lattice_structure: PartialOrder, // constraint ordering
    pub free_energy: Q32,                // thermodynamic stability
    pub formation_ticks: u64,            // how long to crystallize
    pub digest: Hash256,
    pub evidence: EvidenceChain,
}

```

4.5 DSHAE Integration

ECLS signals feed into DSHAE:

- A **Band constraint** on a triangle means DSHAE can expect the mispricing to revert.
- A **Correlation constraint** breaking means regime change — DSHAE should pause.
- A **Topological constraint** ($\beta_1 > 0$ persistent) means a stable arbitrage loop exists.

Listing 11: ECLS -> DSHAE signal

```

pub enum EclsSignal {
    /// A stable constraint lattice exists -- safe to trade within it
    StableLattice { crystal_id: Uuid, affected_pairs: Vec<TradingPair> },
    /// A constraint is breaking -- regime change likely
    ConstraintBreaking { constraint_id: Uuid, severity: Q32 },
    /// New constraint discovered -- expand DSHAE basket
    NewConstraint { candidate: ConstraintCandidate },
}

```

ECLS Rule

ECLS is read-only. It never modifies the HDAG, never trades, never emits execution signals. It discovers and publishes. Downstream systems decide what to do with the information.

5 GUI Additions

Three new tabs in the desktop GUI:

Tab	Contents
Mycelium	MCCE graph visualization: vertices as dots (colored by type, sized by market cap), edges as lines (opacity = correlation strength). 2D force-directed layout. Click vertex to see 5D embedding, edge list, correlation history. Stats: vertex count, edge count, graph density, largest cluster.
Constraints	ECLS constraint list: active constraints sorted by stability. For each: template type, bound entities, satisfaction rate, window size. Lattice Crystal log. Constraint-breaking alerts (highlighted in red).
Knowledge	ISLS substrate overview: storage tier usage (hot/warm/cold), total observations stored, total semantic crystals, graph growth rate chart over time, replay verification status.

6 Sandbox Extensions

Two new sandbox scenarios added to `fixtures/sandbox/`:

File	Duration	What It Tests
<code>scenario_correlation.rec</code>	20000 ticks	Two assets are synthetically correlated for 15000 ticks, then decorrelate. MCCE should detect the correlation as a Hypha, ECLS should discover a Correlation constraint, and ECLS should emit ConstraintBreaking when it ends.
<code>scenario_lattice.rec</code>	20000 ticks	A synthetic 4-asset basket with 3 injected constraints (Band + Ratio + PhaseLock). ECLS should discover all 3 and form a Lattice Crystal.

Expected outcomes (for sandbox PASS/FAIL):

- `scenario_correlation`: MCCE edge count ≥ 1 , ECLS Correlation constraint found before tick 5000, ConstraintBreaking signal emitted after tick 15000.
- `scenario_lattice`: ECLS discovers ≥ 2 of the 3 injected constraints, at least 1 Lattice Crystal emitted.

7 Integration Architecture

Listing 12: Updated macro-cycle with MCCE/ECLS/ISLS

```
// Extended macro-cycle (additions marked with >>>)
1. OBSERVE: ingest market data
2. NORMALIZE: canonical order books
>>> 3. ISLS-PERSIST: write observation to ISLS L0/L1
>>> 4. MCCE-UPDATE: update vertex embeddings, correlation edges
5. EXTRACT: resonance metrics
6. TEMPORAL: advance tri-carrier clocks
7. RESOURCE: check budgets
8. DSHAE/TRUMPET: generate candidates (DSHAE reads MCCE graph)
>>> 9. ECLS-SCAN: if ecls_scan_interval reached, run constraint scanner
>>> 9a. Feed ECLS signals to DSHAE signal filter
10. GATE: evaluate gate stack (Kairos score now includes ECLS stability)
11. EXECUTE or OBSERVE (sniper mode)
12. HEDGE
13. EVIDENCE: append to ISLS evidence chains
14. CALIBRATE
15. INVARIANTS
16. TTCP cascade
>>> 17. MCCE-FRUITING: emit MCCE signals (cluster alerts, new patterns)
>>> 18. ISLS-CONSENSUS: if pending semantic crystals, run consensus
19. ROTATE artifacts
20. STATUS + GUI update
```

Integration Rule

Steps 3, 4, 9, 17, 18 are new. They do NOT replace any existing step. The original 20-step macro-cycle becomes a 24-step macro-cycle. All existing logic (gates, CSP, hedge, TTCP, sniper) is untouched.

8 Configuration

Listing 13: Phase 5 config additions

```
isls:
  enabled: true
  hot_retention_minutes: 60
  warm_retention_days: 7
  cold_compression: zstd
  consensus_threshold: 7500      # Q32: 0.75

mcce:
  enabled: true
  spore_auto_discover: true
  hypha_window_ticks: 100        # correlation window
  hypha_min_rho: 3000            # Q32: 0.30 minimum correlation
  hypha_decay_rate: 50           # Q32: 0.005 per tick
  embedding_update_interval: 10  # re-embed every N ticks
  fruiting_interval: 100         # emit signals every N ticks

ecls:
```

```

enabled: true
scan_interval: 50          # run scanner every N ticks
alpha_min: 8000            # Q32: 0.80 minimum satisfaction
window_size: 500          # ticks for constraint evaluation
templates:
  band: true
  ratio: true
  correlation: true
  granger: false          # computationally expensive, Phase 6
  spectral: false         # requires FFT, Phase 6
  topological: true
  phase_lock: true
lattice_min_constraints: 2  # min constraints for Lattice Crystal
free_energy_threshold: 5000 # Q32: 0.50

```

9 Implementation Order

Step	Component	Done when
5.1	ISLS types	fsr-isls crate. Observation, PersistentGraph, SemanticCrystal, EvidenceChain, TripolarState structs compile. Storage tier logic.
5.2	ISLS persistence	Tiered storage (hot/warm/cold) works. Append, query, compact, migrate between tiers. Existing fsr-chain delegates to ISLS. All Phase 1–4 chain tests still pass.
5.3	MCCE HDAG	fsr-mcce crate. VertexType, 5D embedding, edge creation/update/decay. Unit test: add 10 vertices, embed, verify H_5 coordinates.
5.4	MCCE Spore+Hypha	Spore reads from venue adapters, creates vertices. Hypha computes correlations, creates/prunes edges. Test: synthetic price data → correct correlation edges.
5.5	MCCE Mycelium	TTCP triangulation on vertex cloud. Detect clusters. Test: 3 correlated assets form a cluster (β_0 reduces).
5.6	MCCE integration	Steps 3, 4, 17 added to macro-cycle. MCCE runs alongside engine. Paper mode works with MCCE enabled.
5.7	ECLS types	fsr-ecls crate. 7 constraint templates. ConstraintCandidate. LatticeCrystal.
5.8	ECLS scanner	Scanner evaluates templates against MCCE HDAG. Test: synthetic Band constraint injected → scanner discovers it.
5.9	ECLS lattice	Inverse weaving: group compatible constraints into lattice. Emit LatticeCrystal. Test: 3 constraints → 1 lattice crystal.
5.10	ECLS integration	Step 9 added to macro-cycle. ECLS signals feed DSHAE. Kairos gate score boosted when stable lattice exists.

5.11	GUI Mycelium tab	Force-directed graph rendering. Vertex details on click.
5.12	GUI Constraints tab	Constraint list, lattice crystal log, breaking alerts.
5.13	GUI Knowledge tab	ISLS stats: storage tiers, crystal count, graph growth.
5.14	Sandbox scenarios	2 new scenarios generated and committed. Sandbox PASS/FAIL validation updated.
5.15	Integration test	Full pipeline: paper mode 5000 ticks with MCCE+ECLS+ISLS enabled. All 7 sandbox scenarios pass. All Phase 1–4 tests pass.

10 New Dependencies

Crate	Version	Purpose
petgraph	≥ 0.6	Graph data structure for MCCE HDAG (directed, weighted)

No other new dependencies. ISLS storage uses existing bincode + zstd. ECLS uses no external libraries. GUI uses existing egui.

11 Definition of Done (Phase 5)

1. All Phase 1–4 tests pass unchanged.
2. `cargo clippy -workspace -exclude fsr-gui - -D warnings` passes.
3. ISLS tiered storage: append, query, tier migration, compaction all work. Existing chain semantics preserved.
4. MCCE builds HDAG from venue data: vertices created, embeddings computed, correlations tracked, edges decay.
5. ECLS discovers injected constraints in synthetic data (Band, Correlation, Topological, Phase-Lock templates tested).
6. ECLS emits ConstraintBreaking signal when a synthetic correlation breaks.
7. At least 1 Lattice Crystal emitted in the `scenario_lattice` sandbox.
8. GUI Mycelium tab renders graph with vertex coloring and edge opacity.
9. GUI Constraints tab shows active constraints and lattice crystals.
10. All 7 sandbox scenarios (5 from Phase 4 + 2 new) pass PASS/FAIL validation.
11. Paper mode runs 5000 ticks with MCCE+ECLS+ISLS enabled without panic or invariant violation.
12. No network access required for paper mode or sandbox.

12 Phase 6 Preview (Non-Normative)

- On-chain data ingestion (Ethereum mempool, Solana transactions, DEX pool state).
- Granger causality and spectral coherence templates for ECLS.
- Full persistent homology (Ripser) replacing β_0 -only TTCP.

- Strategy rotation based on ECLS constraint stability.
- Telegram/Signal notification system.
- Multi-config portfolio management in GUI.

End of Phase 5 Specification